Forecasting Wind Power Production in Orkney

Anna Weronika Lekston
IT University of Copenhagen
awle@itu.dk
https://github.com/LSDA-BDM/assignment-mlflow-Annaa73.git
Large Scale Data Analysis (BSLADSA1KU)

BSc in Data Science

March 2026

1 Introduction

The goal of this project is to forecast wind power production in the Orkney archipelago using historical wind power generation data and weather forecasts. Reliable forecasting of renewable energy production is an important task, as it plays an big role in maintaining grid stability and enabling effective planning of energy resources.

The objective of the assignment is not only to build a predictive model but also to demonstrate good practices across the full machine learning lifecycle, including data engineering, preprocessing pipelines, experiment tracking, and model deployment. The final system consists of a reproducible pipeline that processes raw data, trains machine learning models, and makes the trained model available for generating predictions.

2 Data Alignment

The project uses two primary datasets: a `weather.csv` forecast dataset containing meteorological variables such as wind speed and wind direction, and a `power.csv` generation dataset containing minute-level wind power production measurements. Additionally, a `future.csv` dataset is provided for generating predictions using unseen weather forecasts.

The two datasets have different temporal resolutions. The weather data is recorded every three hours, while the power production data is recorded every minute. To combine these datasets, a temporal alignment strategy is required.

Two approaches were considered:

- Upsampling the weather dataset to minute resolution using forward filling
- Downsampling the power dataset to the three-hour weather resolution

After exploratory analysis, downsampling the power dataset was selected. Although this approach reduces the number of observations, it preserves the physical consistency of the weather measurements and avoids introducing artificial repeated values created by forward filling. The power dataset was therefore resampled to a three-hour frequency using mean aggregation.

After resampling, the two datasets were merged using an inner join on the timestamp. Rows with missing target values were removed to ensure valid training samples.

A small number of irregular time gaps were observed in the weather dataset (one six-hour gap and one twelve-hour gap). Given the overall dataset size, these gaps were considered negligible and were handled during preprocessing.

This alignment strategy results in a consistent dataset where each weather observation corresponds to a single aggregated power production value.

3 Data Preprocessing Pipeline

All preprocessing operations were implemented using a `scikit-learn` pipeline to ensure reproducibility and consistent transformations during both training and inference.

3.1 Time-based Train-Test Split

Because the dataset represents a time series, a chronological train–test split was applied before any preprocessing steps. The last 20% of the observations were reserved as a final test set, while the earlier 80% were used for model training and validation. Performing the split before preprocessing ensures that no information from the test set influences feature transformations or model training.

3.2 Time Series Split

Model selection was performed only on the training portion of the data. For this purpose, `TimeSeriesSplit` cross-validation was used. This method generates multiple training and validation folds while preserving the temporal ordering of the observations, meaning that training data always precedes validation data in time. This approach prevents information leakage from future observations into the past.

The preprocessing pipeline was therefore fitted only on the training data within each cross-validation fold and then applied to the corresponding validation fold. After selecting the best-performing model and hyperparameters, the pipeline was refitted on the entire training dataset and finally evaluated on the held-out test set.

3.3 Feature Engineering

Several domain-informed feature transformations were applied in order to better represent the physical relationships between weather conditions and wind power production.

Wind Direction Encoding

Wind direction is a circular variable. A naive numerical encoding would treat directions such as 0° and 360° as distant values, even though they represent the same physical direction. Similarly, directions that are geographically close (e.g. North and North-East) would appear numerically distant if encoded

using categorical indices.

To preserve the circular structure of the variable, wind direction was encoded using sine and cosine transformations. This representation avoids artificial discontinuities and preserves the geometric relationships between directions. Santos [2025]

In addition, the sine and cosine components were multiplied by wind speed to obtain horizontal and vertical wind vector components (u and v). This representation reflects the physical nature of wind as a vector quantity and captures both magnitude and direction simultaneously. Such a representation allows the model to better capture how turbine output depends on both wind speed and direction.

It also prevents problems that arise when applying numerical operations such as interpolation or mean imputation (in the later pipeline step) directly to angular values. Because wind direction is a circular variable, standard arithmetic operations can produce physically incorrect results. For example, the numerical mean of 350° and 10° is 180°, which corresponds to the opposite direction of the true average. By representing wind direction using sine and cosine components, these circular relationships are preserved and interpolation or imputation operations produce geometrically consistent results.

Temporal Features

Exploratory data analysis revealed a clear daily pattern in the average power production across the dataset as shown in Figure 1. Because electricity demand and atmospheric conditions often vary systematically throughout the day, capturing this periodic behaviour can improve predictive performance.

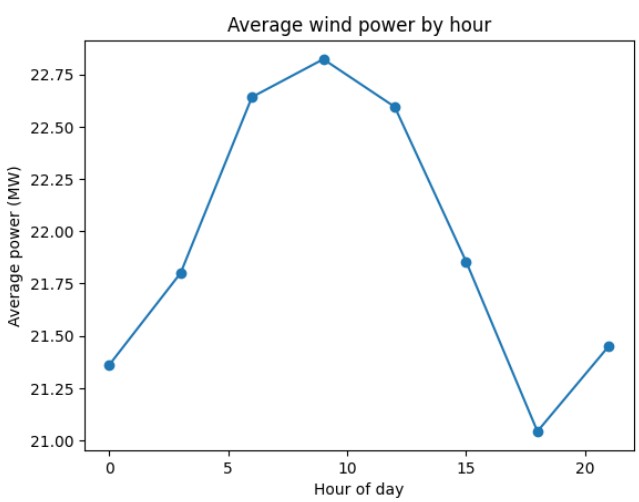

Figure 1: Representation of daily patterns.

To represent this daily cycle, the hour of the day was extracted from the timestamp and encoded using sine and cosine transformations. As with wind direction, time-of-day is a periodic variable. Encoding it using circular transformations prevents artificial discontinuities between hours such as 23:00 and 00:00, which are temporally adjacent but numerically distant if encoded using integers.

Wind Speed Transformation

The relationship between wind speed and power production is strongly nonlinear as we can see in the Figure 2.

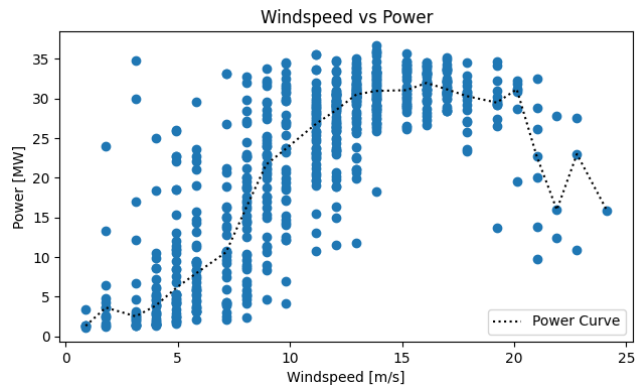

Figure 2: Relationship between the produced power and the wind speed.

According to basic wind energy theory and Betz's law Wikipedia contributors [2026], the theoretical power available in wind is proportional to the cube of wind speed:

$$P \propto v^3$$

This relationship arises because the mass flow rate of air through the turbine rotor is proportional to wind speed, while the kinetic energy of the wind is proportional to the square of wind speed. As a result, the total available power scales with the cube of wind velocity.

To reflect this physical relationship, a cubic transformation of wind speed was introduced as an additional feature (`Speed_cubed`). Including this feature allows the models to capture the nonlinear dependency between wind speed and generated power more effectively.

3.4 Handling Missing Values

Missing values were handled using a two-step strategy combining interpolation and imputation.

First, linear interpolation was applied to fill small temporal gaps in the data. Linear interpolation estimates missing values by connecting the two nearest observed values with a straight line and assigning intermediate values along that line. This approach is well suited for time-series data where observations are ordered in time and changes between consecutive measurements are typically smooth.

For meteorological variables such as wind speed and wind direction components, this assumption is reasonable because atmospheric conditions usually evolve gradually rather than exhibiting instantaneous large jumps between measurements. Therefore, interpolation allows missing observations to be reconstructed using the information from neighboring timestamps while preserving the temporal structure of the data.

However, interpolation cannot handle all missing cases. In particular, it cannot estimate values at the beginning or end of the dataset, or across large missing blocks where no surrounding observations are available. To address any remaining missing values after interpolation, a mean imputation step was applied using an `Imputer` with "mean" approach. This method replaces any remaining missing entries with the average value of the corresponding feature computed from the training data.

Combining interpolation with mean imputation ensures that the dataset contains no missing values while preserving the temporal continuity of the meteorological variables.

3.5 Feature Scaling

Numerical features were standardized using a `StandardScaler`. Scaling is particularly important for distance-based models such as K-Nearest Neighbors and improves numerical stability for linear models.

Although tree-based models are generally insensitive to feature scaling, scaling was retained in the pipeline for consistency across all models.

4 Model Training and Evaluation

After preprocessing and aligning the datasets, several regression models were trained to predict wind power generation. The models were chosen to represent diverse modeling assumptions:

- **Linear Regression:** Simple and interpretable baseline model.
- **K-Nearest Neighbors (KNN):** Distance-based model capturing patterns in nearby feature space.
- **Random Forest:** Ensemble of decision trees trained with bagging, capturing complex non-linear interactions and robust to outliers.
- **Gradient Boosting:** Sequential ensemble method that iteratively corrects previous errors, often yielding higher accuracy for structured tabular data.

4.1 Hyperparameter Tuning

Hyperparameters were optimized using **grid search** with five-fold **time series cross-validation** to preserve chronological order and prevent data leakage. All models were trained within the same preprocessing pipeline:

- Wind direction encoded as vector components (u, v)
- Time-of-day features encoded with sine-cosine transformations
- Cubic transformation of wind speed to capture non-linear effects
- Imputation of remaining missing values using feature-wise mean
- Standard scaling of features

The best hyperparameters found for each model are summarized in Table 1.

Model	Hyperparameters	Values
Linear Regression	—	—
KNN	Number of neighbors	5
Random Forest	Max depth	10
	Min samples split	5
	Number of estimators	200
Gradient Boosting	Learning rate	0.01
	Max depth	3
	Number of estimators	200

Table 1: Best hyperparameters for each regression model after grid search.

4.2 Model Performance

Models were evaluated using multiple regression metrics: **MAE**, **RMSE**, R^2 **score**, and **Explained Variance**. The performance of the best configuration for each model under cross-validation is shown in Table 2 and Table 3.

Model	ExplVar	MAE (MW)
Linear Regression	-0.16	9.47
KNN (n=5)	0.42	6.43
Gradient Boosting	0.59	5.09
Random Forest	0.56	4.70

Table 2: Cross-validation performance: Explained Variance and MAE.

Model	RMSE (MW)	R^2
Linear Regression	11.51	-0.59
KNN (n=5)	8.30	0.21
Gradient Boosting	6.47	0.51
Random Forest	6.42	0.50

Table 3: Cross-validation performance: RMSE and R^2.

Key observations include:

- **Linear Regression** underperforms, indicating that the relationship between weather features and power output is non-linear.
- **KNN** captures local dependencies better but lacks the capacity to model global non-linear patterns.
- **Random Forest** and **Gradient Boosting** achieve the best performance, modeling complex feature interactions effectively.
- Random Forest slightly outperformed Gradient Boosting.

4.3 Final Model and Test Evaluation

The final selected model was **Random Forest** (max depth 10, min samples split 5, 200 estimators). Retraining on the entire training set and evaluating on the chronological test set yielded:

- Explained Variance = 0.76
- MAE = 3.86 MW
- RMSE = 5.51 MW
- R^2 = 0.75

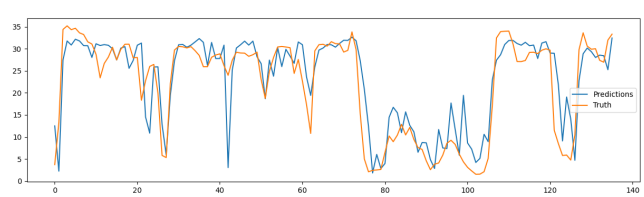

Figure 3: Predicted vs True values using the final model.

The slightly better test performance compared to cross-validation might be caused because of the more stable distribution in the last 20% of the dataset. The model was then applied to the `future.csv` dataset to generate predictions,

which were saved as `future_predictions_final.csv`. All experiments, including hyperparameters, metrics, and artifacts, were tracked using **MLflow** for reproducibility and deployment.

5 Experiment Tracking with MLflow

MLflow was used to track experiments and manage model development.

For each experiment run, the following information was logged:

- Model hyperparameters
- Evaluation metrics
- Trained model artifacts
- Pipeline configuration

MLflow experiments were organized by model type, allowing easy comparison between different model variants. The MLflow UI was used to visualize and compare results across runs, enabling selection of the best-performing model.

6 Model Serving

The final Random Forest model was registered using the MLflow Model format. This format stores the trained model together with the full preprocessing pipeline and the environment dependencies required to run it.

After registration, the model was deployed using MLflow's model serving functionality. This creates a small web service that allows the model to receive input data and return predictions.

This setup ensures that the same preprocessing steps used during training are also applied when the model receives new data for prediction, which guarantees consistency between training and inference.

7 Reproducibility with MLflow Projects

To ensure reproducibility, the training pipeline was packaged as an MLflow Project. The project includes:

- A training script implementing the full pipeline
- An `MLproject` configuration file
- A conda environment specification defining all dependencies

This allows the entire experiment to be executed on another machine using a single command:

```
mlflow run .
```

This guarantees consistent environments and reproducible results.

8 Reflection and Limitations

While the proposed pipeline and models are able to capture some of the relationships between weather conditions and power generation, several limitations of the dataset and modeling approach should be acknowledged.

8.1 Limited Dataset Size

One important limitation is the relatively small size of the dataset. The available observations cover only the winter months (December–March), evading information about wind conditions during other seasons of the year.

Wind patterns often vary significantly across seasons due to changes in atmospheric circulation, temperature gradients, and large-scale weather systems. As a result, models trained only on winter data may not generalize well to other parts of the year. Additionally, the limited time coverage makes it difficult to detect longer-term patterns such as monthly or seasonal cycles in wind power production.

A larger dataset spanning multiple years and all seasons would allow the model to capture broader temporal dynamics and potentially improve forecasting performance.

8.2 Temporal Resolution Mismatch

Another challenge arises from the mismatch in temporal resolution between the two datasets used in the project. The weather.csv dataset provides meteorological forecasts every three hours, while the power.csv production dataset records measurements every minute.

To align these datasets, the power data was downsampled to match the three-hour frequency of the weather data. While this approach preserves the integrity of the weather measurements and avoids introducing artificial repeated values, it also leads to a substantial loss of information. High-frequency fluctuations in power production that occur at shorter time scales are therefore not captured by the final training dataset.

An alternative approach could involve using higher-resolution weather forecasts or applying more advanced interpolation techniques when upsampling the weather variables. This could allow the model to exploit the richer temporal structure present in the minute-level power production data.

8.3 Absence of Lag Features

In the current modeling approach, each observation is treated independently, meaning that the model predicts power output using only the weather conditions observed at the same timestamp. However, wind power generation is known to exhibit strong temporal autocorrelation: the current power output is often strongly influenced by the recent history of wind conditions and turbine operation.

Because wind conditions typically evolve gradually over time, recent observations contain valuable information about short-term trends and persistence in power generation. Incorporating lagged values of the target variable (for example, power production in the previous time steps) would allow the model to explicitly capture these temporal dependencies.

Lag features could therefore transform the model into a partially autoregressive forecasting system, potentially improving predictive accuracy. Previous research in wind power forecasting has shown that including lagged features often leads to improvements in error metrics such as MAE and RMSE Rathnayake [2025].

8.4 Normalization by Installed Capacity

Another potential improvement concerns the interpretation of evaluation metrics. In this project, error metrics such as MAE and RMSE are reported directly in megawatts (MW). While these metrics quantify prediction accuracy, their interpretation depends on the scale of the wind farm.

For example, an MAE of 5 MW may represent a relatively small error for a large wind farm with a capacity of 100 MW, but it could be substantial for a smaller installation with a capacity of 10 MW. To address this issue, evaluation metrics can be normalized by the installed generation capacity P_{nom} of

the wind farm.

Normalized metrics can be defined as:

$$nMAE = \frac{MAE}{P_{\text{nom}}}, \qquad nRMSE = \frac{RMSE}{P_{\text{nom}}}$$

Such normalization produces dimensionless error measures that make model performance easier to interpret and compare across different wind farms, datasets, or time periods. For example, reporting that the forecast error corresponds to approximately 10% of the maximum production capacity is often more meaningful for grid operators and energy planners.

8.5 Future Research Directions

Future work could extend the current system by incorporating additional meteorological variables, higher-resolution weather forecasts, or advanced time-series models. Methods such as Multi-Layer Perceptron (MLP) and Long Short-Term Memory (LSTM) networks could potentially capture more complex temporal dependencies Columbus [2025].

Combining improved data availability with more advanced modeling techniques may further enhance the accuracy and robustness of wind power forecasting systems.

9 Screen captures from the MLflow UI

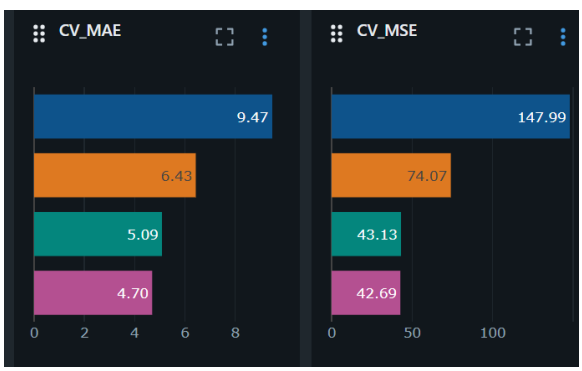

Figure 4: Visual comparison of different models' metrics using the Mlflow's Chart view.

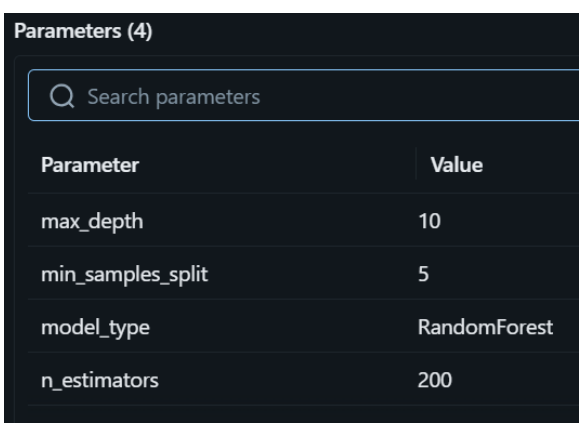

Figure 5: Parameters of the best model saved in MLflow.

10 Code Snippets

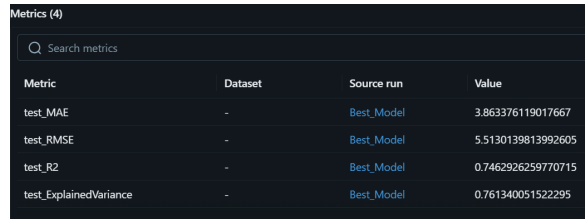

Figure 6: Evaluation metrics of the best trained model presented in MLflow.

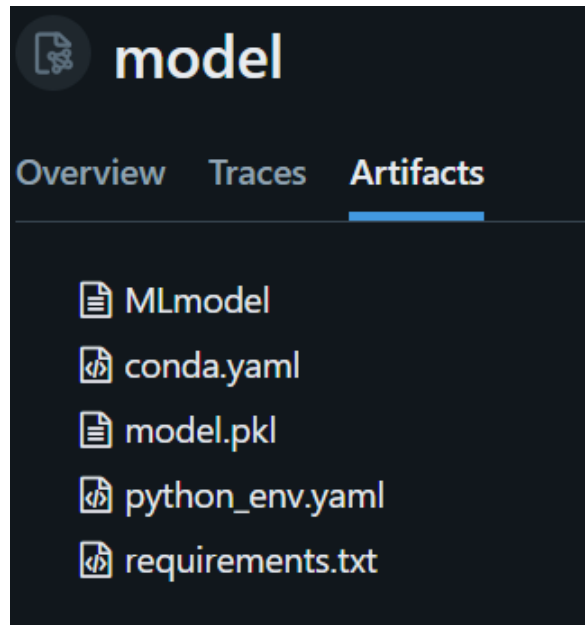

Figure 7: Saved artifacts of the best model in the MLflow.

References

T. A. Rajaperumal C. Christopher Columbus. Enhanced wind power forecasting using machine learning, deep learning models and ensemble integration. 2025. URL `https://www.nature.com/articles/s41598-025-05250-3`.

Rashmi Semasinghe Upaka Rathnayake Rathnayake, Jeevani Jayasinghe. Predicting short-term wind power generation at musalpetti wind farm: Model development and analysis. 2025. URL `https://www.techscience.com/CMES/v143n2/61455/html`.

Gustavo Santos. Is your model time-blind? the case for cyclical feature encoding. 2025. URL `https://towardsdatascience.com/is-your-model-time-blind-the-case-for-cyclical-feature`

Wikipedia contributors. Betz's law, 2026. URL `https://en.wikipedia.org/wiki/Betz's_law`.

```
● @Annaa73 →/workspaces/assignment-mlflow-Annaa73 (main) $ curl -X POST \$ curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"dataframe_split": {"columns": ["time","Direction","Lead_hours","Source_time","Speed","ANM","Non-ANM"]
  , "data":[["2022-03-11 16:00:00","SE",4,1646996444,15.2,0,0]]}}' \
  http://127.0.0.1:1234/invocations
{"predictions": [31.452497361450934]}@Annaa73 →/workspaces/assignment-mlflow-Annaa73 (main) $
```

Figure 8: Sending prediction request.

```python
data_3hrs["hour"] = data_3hrs["time"].dt.hour

data_3hrs.groupby("hour")["Total"].mean().plot(marker="o")
plt.title("Average wind power by hour")
plt.xlabel("Hour of day")
plt.ylabel("Average power (MW)")
plt.show()
# daily pattern present
```

Figure 9: Plotting daily patterns.

```python
# downsample power_df to to 3-hour resolution
# simplest but we loose infomration, loses information

power_3h = (
    power_df
    .set_index("time")
    .resample("3H")
    .mean()
    .reset_index()
)

data_3hrs = pd.merge(wind_df, power_3h, on="time", how="inner")

# remove rows where power is missing
data_3hrs = data_3hrs.dropna(subset=["Total"])
```

Figure 10: Datasets alignment.

```python
class FeatureEngineering(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        X = X.copy()
        compass_to_deg = { "N":0,"NNE":22.5,"NE":45,"ENE":67.5,
                           "E":90,"ESE":112.5,"SE":135,"SSE":157.5,
                           "S":180,"SSW":202.5,"SW":225,"WSW":247.5,
                           "W":270,"WNW":292.5,"NW":315,"NNW":337.5 }
        if "Direction" in X.columns:
            X["angle"] = X["Direction"].map(compass_to_deg)
            X["angle"] = np.deg2rad(X["angle"])
            X["u"] = -X["Speed"] * np.sin(X["angle"])
            X["v"] = -X["Speed"] * np.cos(X["angle"])
        if "Speed" in X.columns:
            X["wind_speed_cubed"] = X["Speed"]**3
        if "time" in X.columns:
            hour = pd.to_datetime(X["time"]).dt.hour
            X["hour_sin"] = np.sin(2*np.pi*hour/24)
            X["hour_cos"] = np.cos(2*np.pi*hour/24)
        drop_cols = ["Direction", "Speed", "angle", "time", "Lead_hours", "Source_time", "ANM", "Non-ANM"]
        X = X.drop(columns=[c for c in drop_cols if c in X.columns])
        return X
```

Figure 11: Class with all transformations conducted on the features.

```python
class InterpolateData(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        X = pd.DataFrame(X)
        return X.interpolate(method="linear", limit_direction="both")

class Imputer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        self.means_ = X.mean(numeric_only=True)
        return self
    def transform(self, X):
        X = X.copy()
        return X.fillna(self.means_)
```

Figure 12: Classes with Interpolation and Imputer steps.

```python
# Define models and hyperparameters
experiments = {
    "RandomForest": {
        "model": RandomForestRegressor,
        "params": {
            "n_estimators": [100, 200],
            "max_depth": [10, 20],
            "min_samples_split": [2, 5]
        }
    },
    "GradientBoosting": {
        "model": GradientBoostingRegressor,
        "params": {
            "n_estimators": [100, 200],
            "learning_rate": [0.01, 0.1],
            "max_depth": [3, 5]
        }
    },
    "KNN": {
        "model": KNeighborsRegressor,
        "params": {
            "n_neighbors": [3, 5]
        }
    },
    "LinearRegression": {
        "model": LinearRegression,
        "params": {}
    }
}
```

Figure 13: List of all models and parameters.

```python
# Log the best model (trained on the full training dataset)
with mlflow.start_run(run_name="Best_Model"):

    mlflow.log_param("model_type", best_model_name)
    mlflow.log_params(best_params)

    for k, v in test_metrics.items():
        mlflow.log_metric(f"test_{k}", v)

    # infer model signature
    signature = infer_signature(X_train, best_pipeline.predict(X_train))

    mlflow.sklearn.log_model(
        best_pipeline,
        "model",
        signature=signature,
        registered_model_name="WindPowerForecastModelFinal"
    )

    mlflow.log_artifact("plots/predictions_final.png")
    mlflow.log_artifact("future_predictions_final.csv")

print("Forecasting completed! Predictions saved to future_predictions.csv")
```

Figure 14: Training the final model and last steps in the code.